School of
Management and Law

MDM
ModelOps – Automation & Deployment

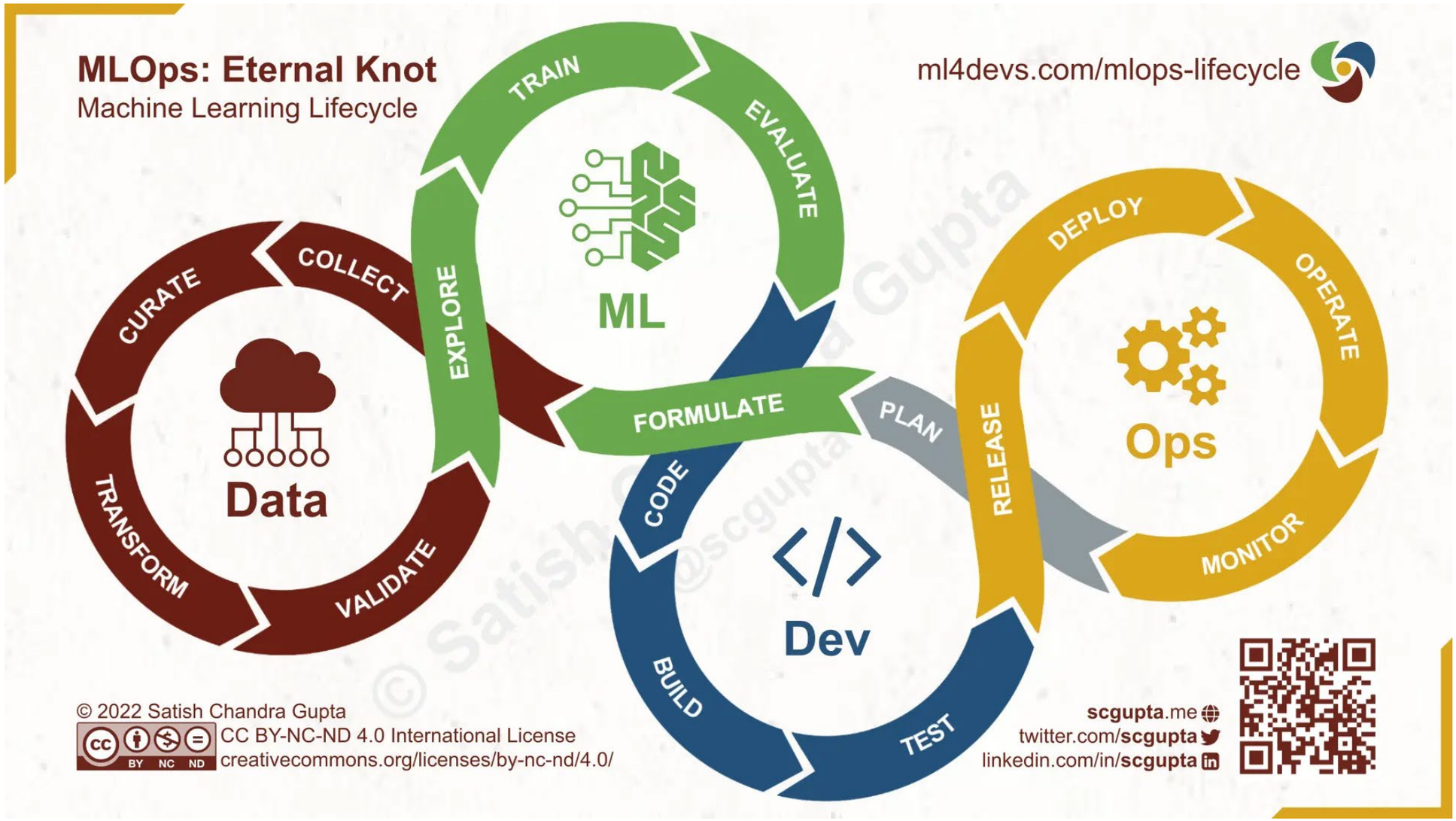
Adrian Moser
adrian.moser@zhaw.ch, FS2026

ModelOps

Viele Buzzwords…

ModelOps
MLOps
DataOps
DevOps
NoOps
BizDevOps

Übersicht


MLOps: Eternal Knot
Machine Learning Lifecycle
ml4devs.com/mlops-lifecycle
TRAIN
EVALUATE
ML
CURATE
COLLECT
EXPLORE
Data
TRANSFORM
VALIDATE
FORMULATE
PLAN
CODE
Dev
BUILD
TEST
RELEASE
DEPLOY
OPERATE
Ops
MONITOR
© 2022 Satish Chandra Gupta
CC BY-NC-ND 4.0 International License
creativecommons.org/licenses/by-nc-nd/4.0/
scgupta.me
twitter.com/scgupta
linkedin.com/in/scgupta

Beispiel «HikePlanner»

Problem

Lösungsidee

Wieviel Zeit wird benötigt?

Modell in Cloud-Speicher ablegen

Azure Blob Storage

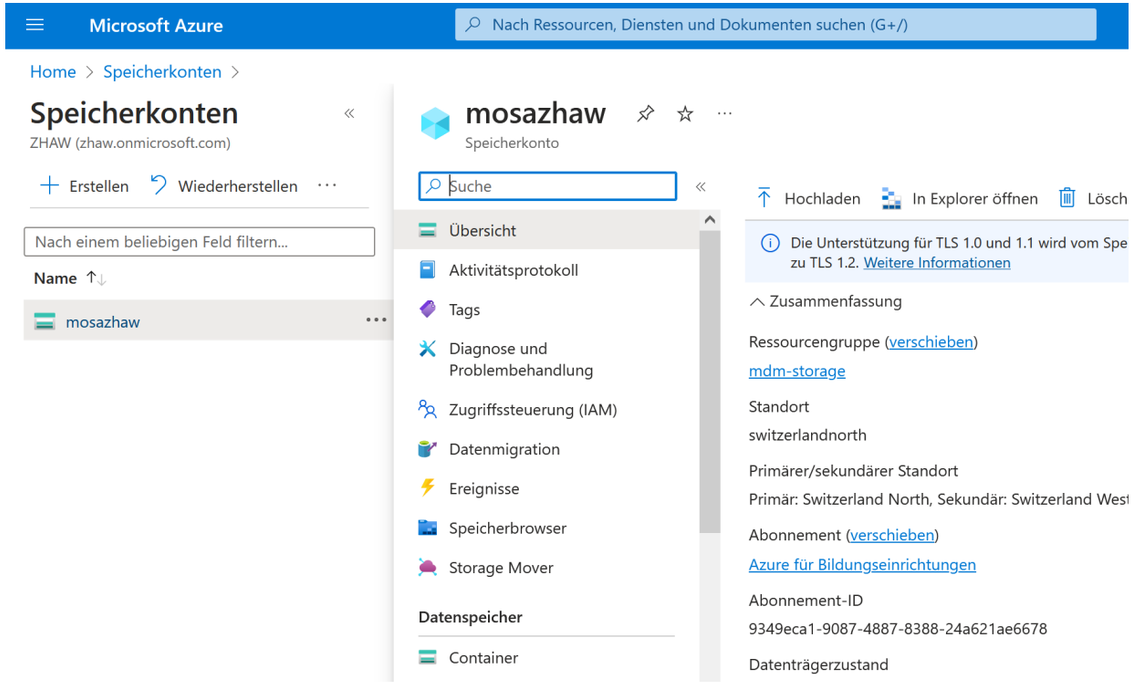
Azure Blob Storage erstellen

Ressourcengruppe erstellen

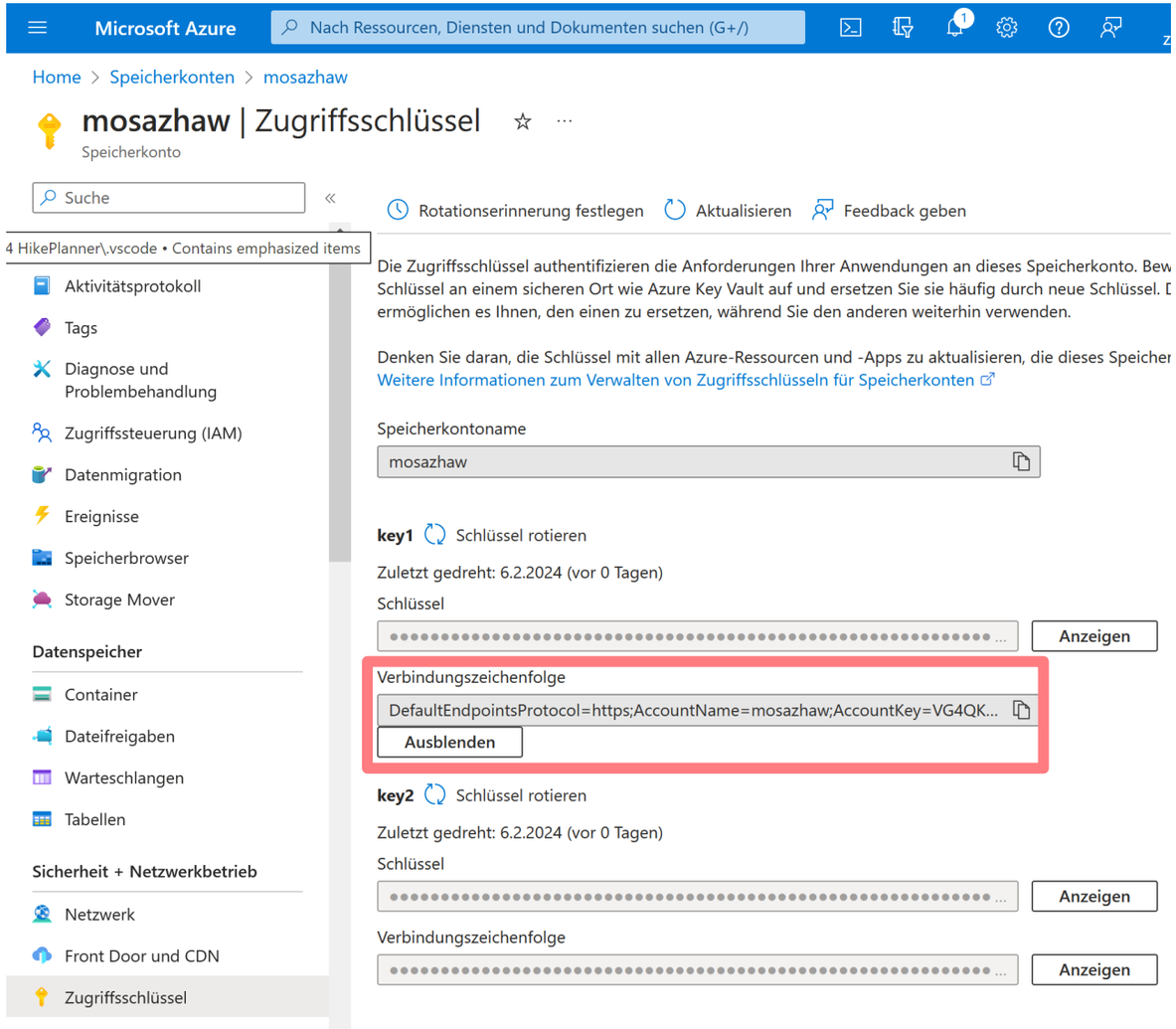
```
az group create --name mdm-storage --location switzerlandnorth
```

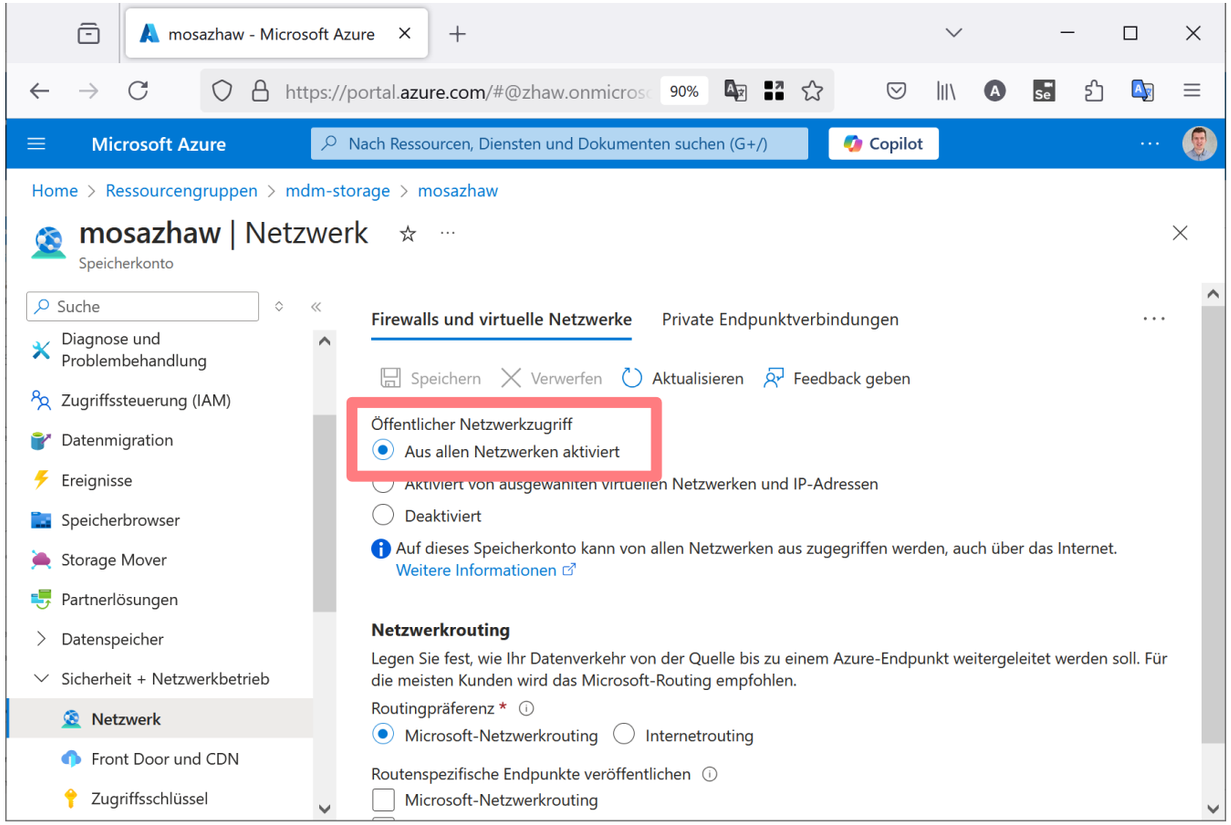
Storage erstellen

```
az storage account create --name mosazhaw
--resource-group mdm-storage --location
switzerlandnorth --sku Standard_RAGRS --
kind StorageV2 --allow-blob-public-access
false
```



Zugriffsschlüssel verwenden / Zugriff aus allen Netzwerken überprüfen

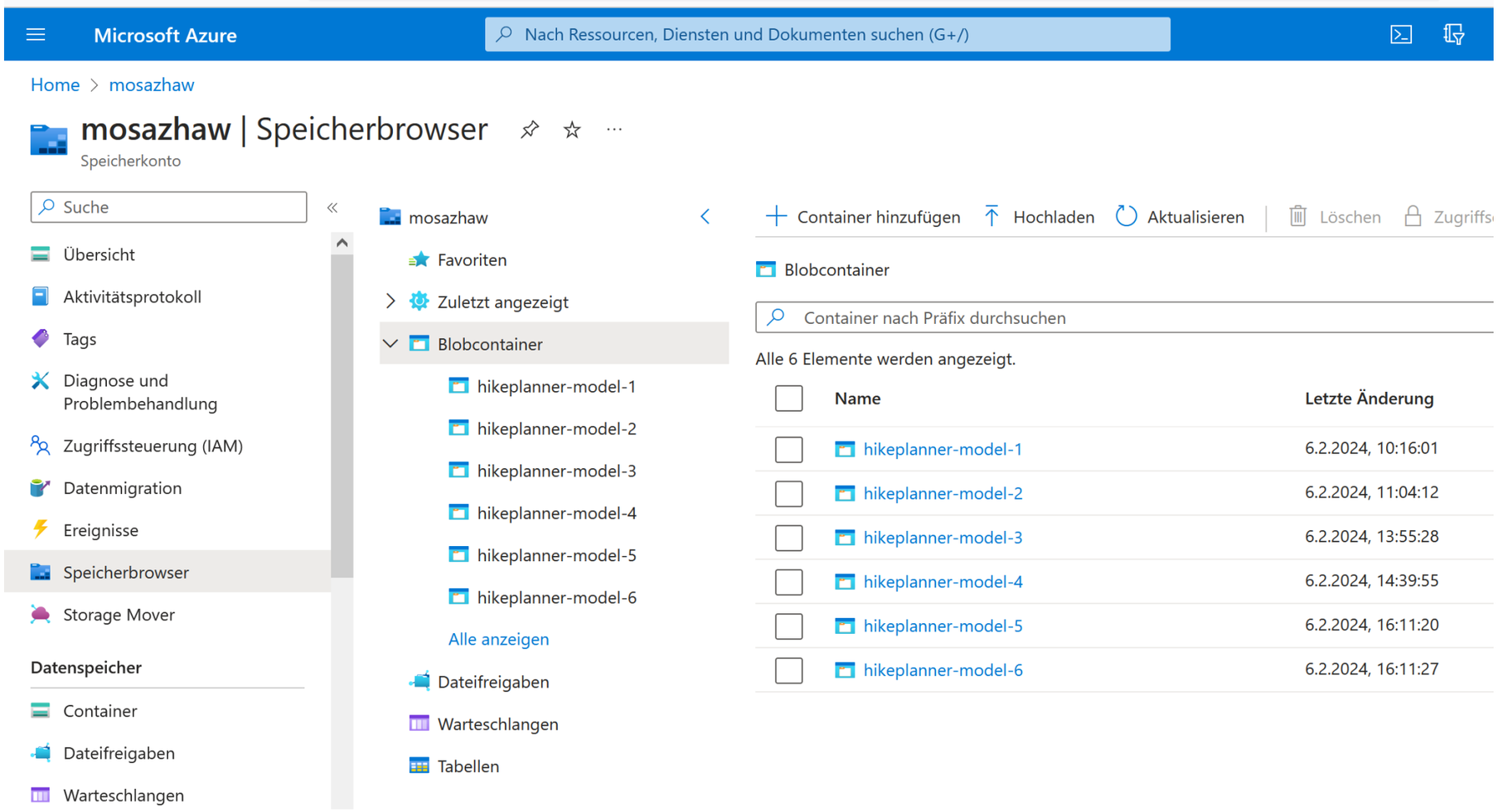




Modell auf Azure Storage speichern

Einfache Versionierung

- Für jedes neue Modell neuen Blobcontainer erstellen
- Suffix: Aufsteigende Nummerierung
- Das neueste Modell im Beispiel ist die Nr. 6



Modell speichern

Terminal, cd model

```
(.venv) C:\HikePlanner\model>python publish_model.py
Missing AZURE_STORAGE_CONNECTION_STRING in .env or in the environment.
```

.env ergänzen

```
# env values
MONGO_DB_CONNECTION_STRING=…
AZURE_STORAGE_CONNECTION_STRING=DefaultEndpointsProtocol=https;AccountName=mosazhaw;AccountKey=***
;EndpointSuffix=core.windows.net
```

Jedes neue Modell wird versioniert abgelegt

```
(.venv) C:\HikePlanner\model>python publish_model.py
Using container: hikeplanner-model-5
Uploading GradientBoostingRegressor.pkl...
Uploading LinearRegression.pkl...
```

Modell verwenden

Flask Service

Svelte Frontend

Frontend

Frontend Build

- Der Build des Svelte-Frontends ist (ausnahmsweise) im Git-Repo enthalten
- Daher entfällt ein «node install» und «vite build», sofern keine Anpassungen gemacht werden

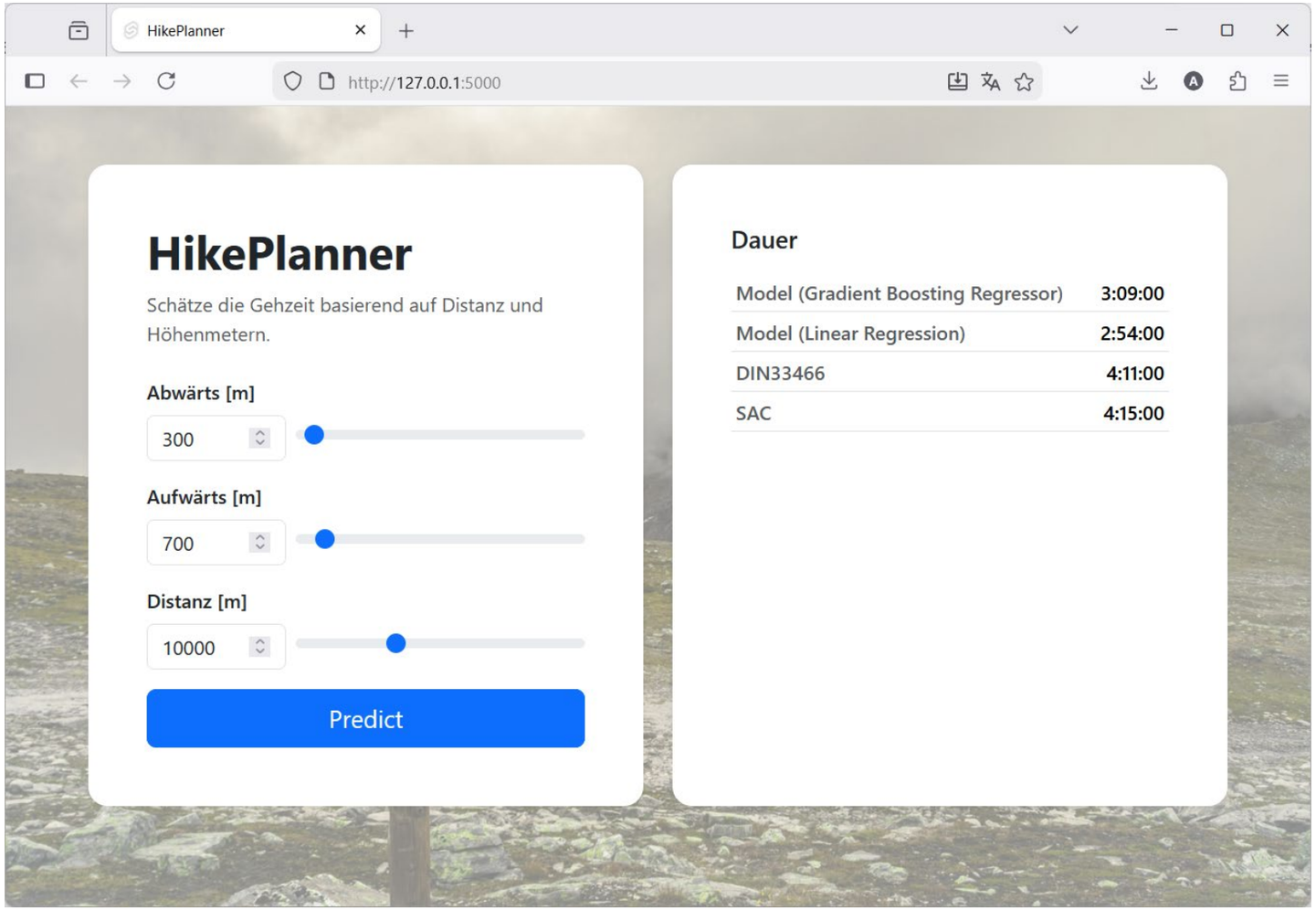
Frontend Start

- Das statisch vorhandene Frontend wird von Flask zur Verfügung gestellt
- Sofern nicht entwickelt wird, muss das Frontend deshalb nicht separat gestartet werden

App lokal starten

Starten mittels VS-Code

Die App ist nun lokal verfügbar



Automatisierung

Ziel der Automatisierung

Ablauf

1. (Neue) Daten verarbeiten
2. Daten in MongoDB speichern
3. Neues Modell erstellen
4. (Kennzahlen des Modells prüfen)
5. Neues Modell in Azure Storage hochladen
6. App kann neues Modell verwenden

GitHub Actions

Konzept

CI/CD-Vorgänge wie Build, Test und Deployment direkt aus dem GitHub-Repository heraus ausführen (sogenannte Workflows). Begrenzte Menge an monatlichen freien Minuten (https://github.com/settings/billing)

Workflow

Ein Workflow besteht aus einem oder mehreren Jobs. Workflows sind in **YAML-Files** (YAML Ain't Markup Language) definiert und werden im .github-Verzeichnis im Repository gespeichert.

Ausführung

Manuell über das GitHub-Webinterface oder bevorzugt beim Ausführen eines Push auf das GitHub-Repository.

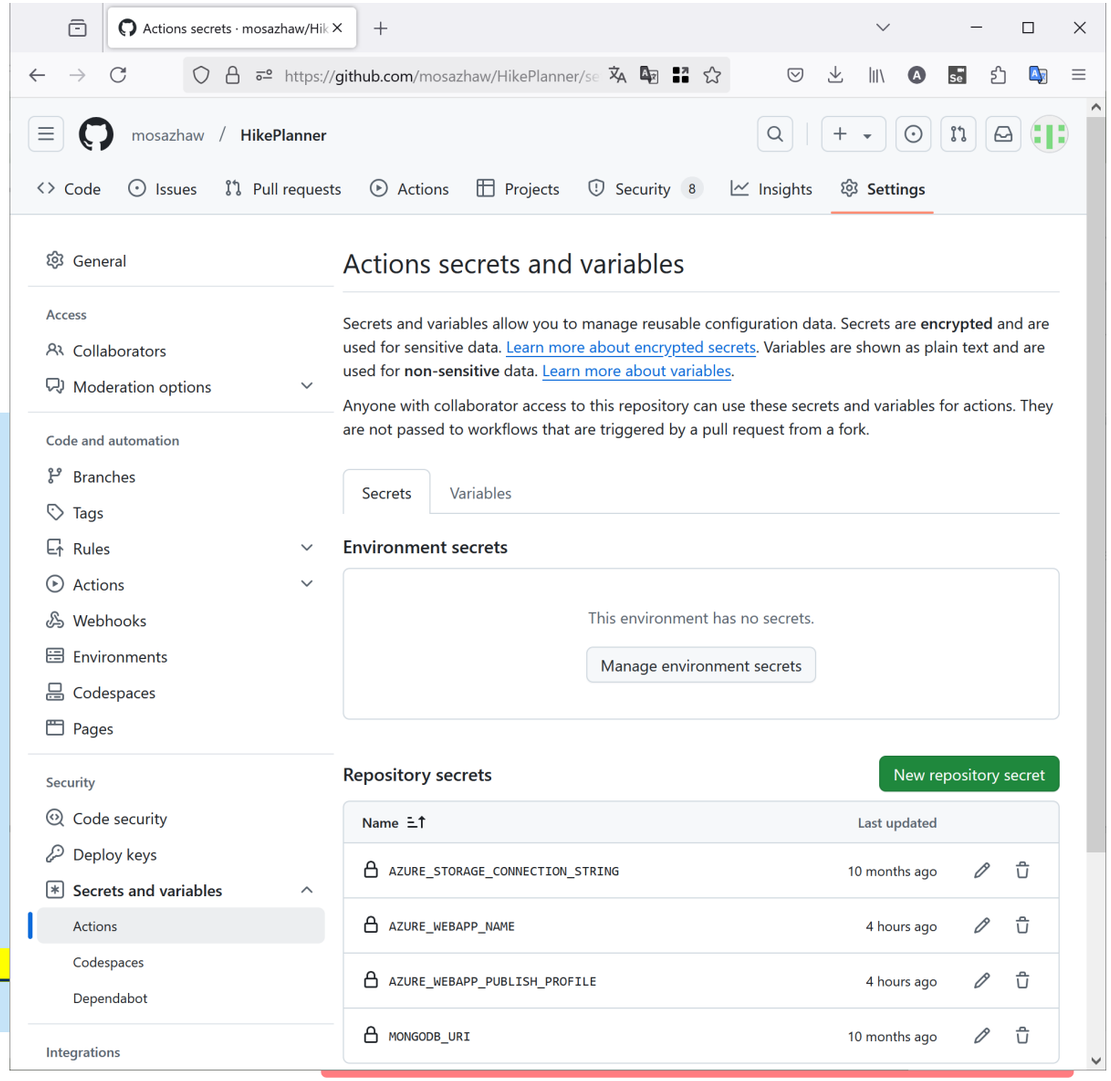
```
name: docker hub push

on:
  push:
    branches:
      - 'main'
```

GitHub Actions

Secrets

GitHub Actions können auch Zugangsdaten wie Passwörter enthalten. Diese werden über Variablen referenziert.

```
- name: upload data to mongodb
  working-directory: data
  env:
    MONGO_DB_CONNECTION_STRING: ${{ secrets.MONGO_DB_CONNECTION_STRING }}
  run: uv run python ./transform-validate.py

- name: build model
  working-directory: model
  env:
    MONGO_DB_CONNECTION_STRING: ${{ secrets.MONGO_DB_CONNECTION_STRING }}
  run: uv run python ./train_model.py

- name: upload model
  working-directory: model
  env:
    AZURE_STORAGE_CONNECTION_STRING: ${{ secrets.AZURE_STORAGE_CONNECTION_
  run: uv run python ./publish_model.py
```



Hinweise zur Umsetzung

GitHub Action Secrets

Folgende Secrets müssen unter https://github.com/XYZ/settings/secrets/actions erfasst werden:

- **AZURE_STORAGE_CONNECTION_STRING**
- **MONGO_DB_CONNECTION_STRING**

Action

Abgelegt im GitHub-Repo unter .github\workflows\model.yml

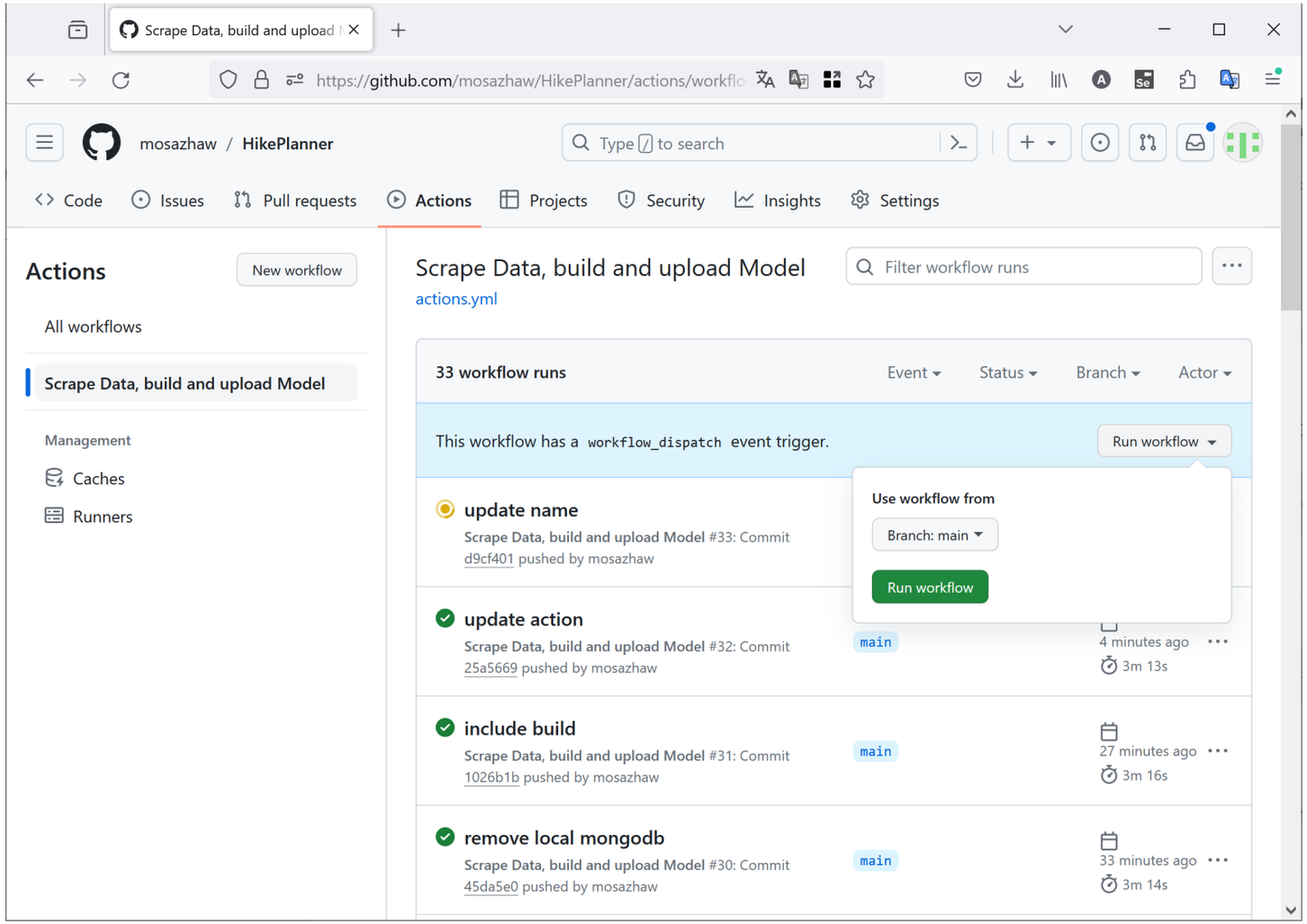
```
name: ModelOps (Update Model)
# Requires GitHub Action Secrets: AZURE_STORAGE_CONNECTION_STRING, MONGO_DB_CONNECTION_STRING

on:
  # push:
  workflow_dispatch:

jobs:
  model:
    runs-on: ubuntu-latest
    steps:

      - name: checkout repo content
        uses: actions/checkout@v3 # checkout the repository content to github runner

      - name: setup python
        uses: actions/setup-python@v4
        with:
          python-version: '3.13.7' # install the python version needed
          cache: 'pip'

      - name: install uv
        uses: astral-sh/setup-uv@v3





...
```

GitHub Action ausführen

Action-Ausführung

Wird in «on»-Bereich festgelegt

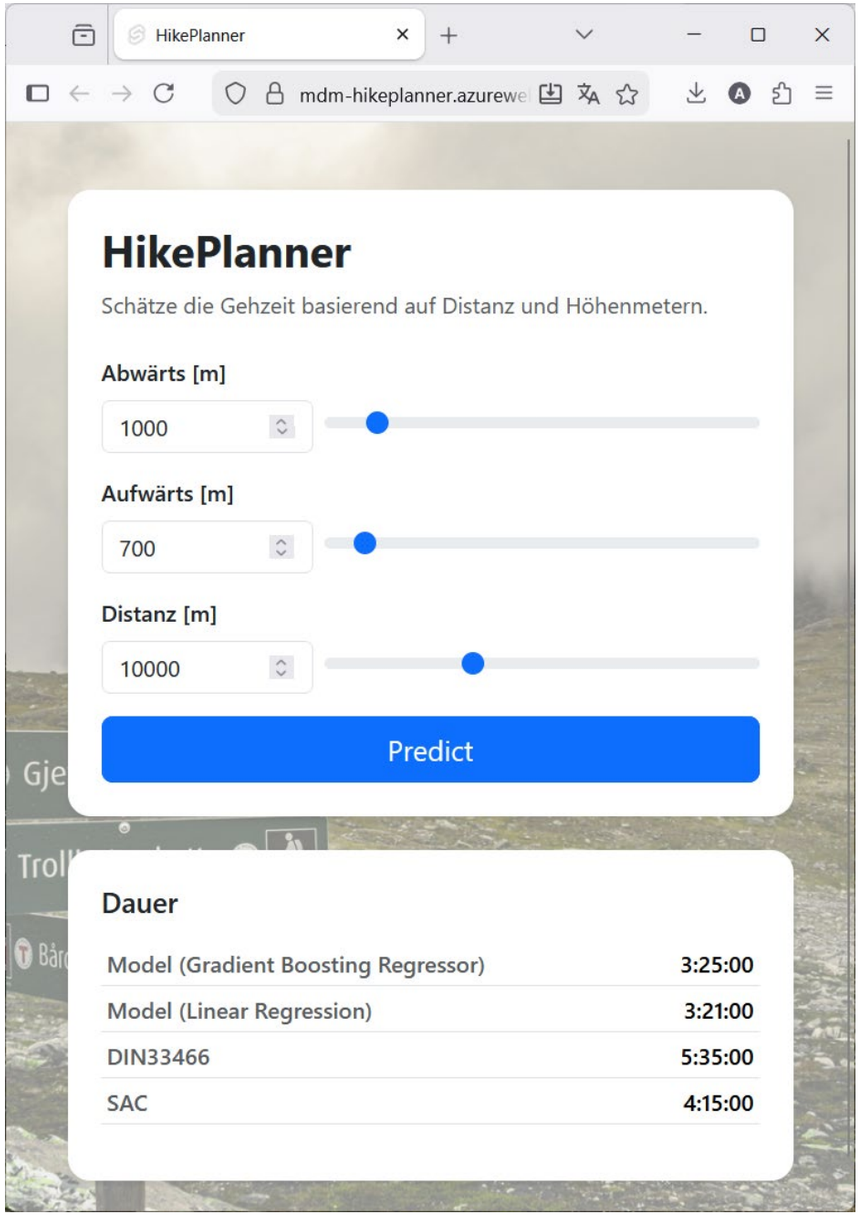
```
name: ModelOps (Build, Deploy)


on:
  push:
  workflow_dispatch:

jobs:
  build:
```



Ausführung

- **push:** bei Push, Angabe Branch möglich
- **workflow_dispatch**: manuell
- nach anderen Actions (siehe HikePlanner)

Deployment


HikePlanner
mdm-hikeplanner.azurewe
HikePlanner
Schätze die Gehzeit basierend auf Distanz und Höhenmetern.
Abwärts [m]
1000
Aufwärts [m]
700
Distanz [m]
10000
Predict
Dauer
Model (Gradient Boosting Regressor) 3:25:00
Model (Linear Regression) 3:21:00
DIN33466 5:35:00
SAC 4:15:00

Dockerfile

Basis
Python 3.13.7

Frontend
wird als gebaut
vorausgesetzt

uv
Installation mit pip
uv sync

```
FROM python:3.13.7

# Install uv and dependencies
WORKDIR /usr/src/app
COPY pyproject.toml uv.lock ./

RUN pip install uv
RUN uv sync --frozen --no-dev
ENV PATH="/usr/src/app/.venv/bin:$PATH"

# Copy application files
COPY backend/app.py backend/app.py
COPY frontend/build frontend/build

# Docker Run Command
EXPOSE 80
ENV FLASK_APP=/usr/src/app/backend/app.py
CMD [ "python", "-m" , "flask", "run", "--host=0.0.0.0", "--port=80"]
```

Wie wird AZURE_STORAGE_CONNECTION_STRING übergeben?

Build Docker Image

Docker Image bauen

```
docker build -t mosazhaw/hikeplanner .
```

Lokal ausführen (mit Übergabe des Connection Strings, Anführungszeichen je nach Shell)

```
docker run --name hikeplanner -e AZURE_STORAGE_CONNECTION_STRING='***' -p 9001:80 -d
mosazhaw/hikeplanner
```

Push auf Docker Hub

```
docker push mosazhaw/hikeplanner:latest
```

Manuelles ACI Deployment

Vorbedingungen:

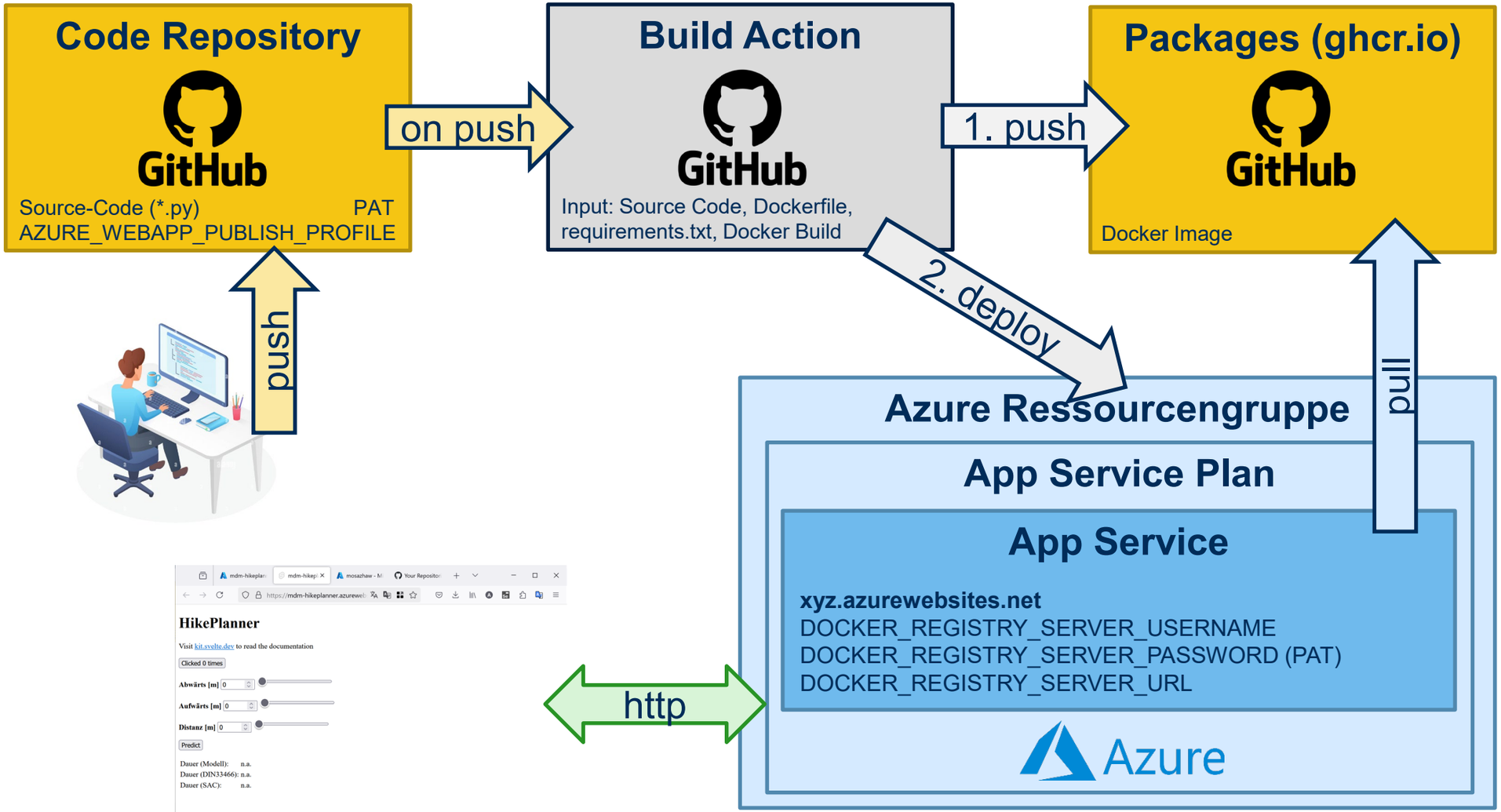
```
az group create --name mdm-hikeplanner --location switzerlandnorth
```

ACI Deployment:

```
az container create --resource-group mdm-hikeplanner --name mdm-hikeplanner --image mosazhaw/mdm-
hikeplanner:latest --dns-name-label mdm-hikeplanner --ports 80 --environment-variables
'AZURE_STORAGE_CONNECTION_STRING'='DefaultEndpointsProtocol=https;AccountName=mosazhaw;AccountKey=************
************************************;EndpointSuffix=core.windows.net'
```

URL: http://mdm-hikeplanner.switzerlandnorth.azurecontainer.io

Automatisches Deployment: Überblick



Automatisches Deployment: GitHub Action

Web App **einmalig** einrichten (beliebiges Image) und Umgebungsvariable setzen:

```
az group create --name mdm-hikeplanner --location switzerlandnorth
```

```
az appservice plan create --name mdm-hikeplanner --resource-group mdm-hikeplanner --sku F1 --is-linux
```

```
az webapp create --resource-group mdm-hikeplanner --plan mdm-hikeplanner --name mdm-hikeplanner --container-image mosazhaw/hikeplanner:latest
```

```
az webapp config appsettings set --resource-group mdm-hikeplanner --name mdm-hikeplanner --settings AZURE_STORAGE_CONNECTION_STRING=***
```

GitHub Action-Secrets hinterlegen

- AZURE_WEBAPP_PUBLISH_PROFILE: Profil von Azure Portal laden (Web App-Übersicht)
- AZURE_WEBAPP_NAME: Name der App

Privates GitHub-Repo

- DOCKER_REGISTRY_SERVER_USERNAME: GitHub-Benutzername
- DOCKER_REGISTRY_SERVER_PASSWORD: GitHub Personal Access Token (PAT)
- DOCKER_REGISTRY_SERVER_URL: https://ghcr.io

Azure Publish Profile

SCM-Standardauthentifizierung aktivieren

Microsoft Azure | Nach Ressourcen, Diensten und Dokumenten suchen (G+/)

Home > mdm-hikeplanner

mdm-hikeplanner | Allgemeine Einstellungen
Web-App

Suche

- Übersicht
- Aktivitätsprotokoll
- Zugriffssteuerung (IAM)
- Tags
- Diagnose und Problembehandlung
- Microsoft Defender for Cloud
- Ereignisse (Vorschau)
- Protokollstream
- Ressourcenschnellansicht
- Bereitstellung
  - Bereitstellungsslots
  - Bereitstellungscenter
- Einstellungen
  - Umgebungsvariablen
  - Konfiguration (Vorschau)

Allgemeine Einstellungen | Stapeleinstellungen | Integritätsprüfung | Pfadzuordnungen | Fehlerseiten

Aktualisieren | Vorschau beenden

Plattformeinstellungen

Anmeldeinformationen für die SCM-Standardauthentifizierungsveröffentlichung ☑

Anmeldeinformationen für die FTP-Standardauthentifizierungsveröffentlichung ☐

FTP-Status: Nur FTPS

HTTP-Version: 2.0

HTTP 2.0-Proxy: Aus

SSH: Das Ändern der SSH-Einstellung wird für Ihre ausgewählte Stapelversion nicht unterstützt.

Always On: Always On erfordert App Service-Plan "Basic" oder höher

Profil laden

Microsoft Azure | Nach Ressourcen, Diensten und Dokumenten suchen (G+/) | Copilot

Home >

mdm-hikeplanner
Web-App

Suche

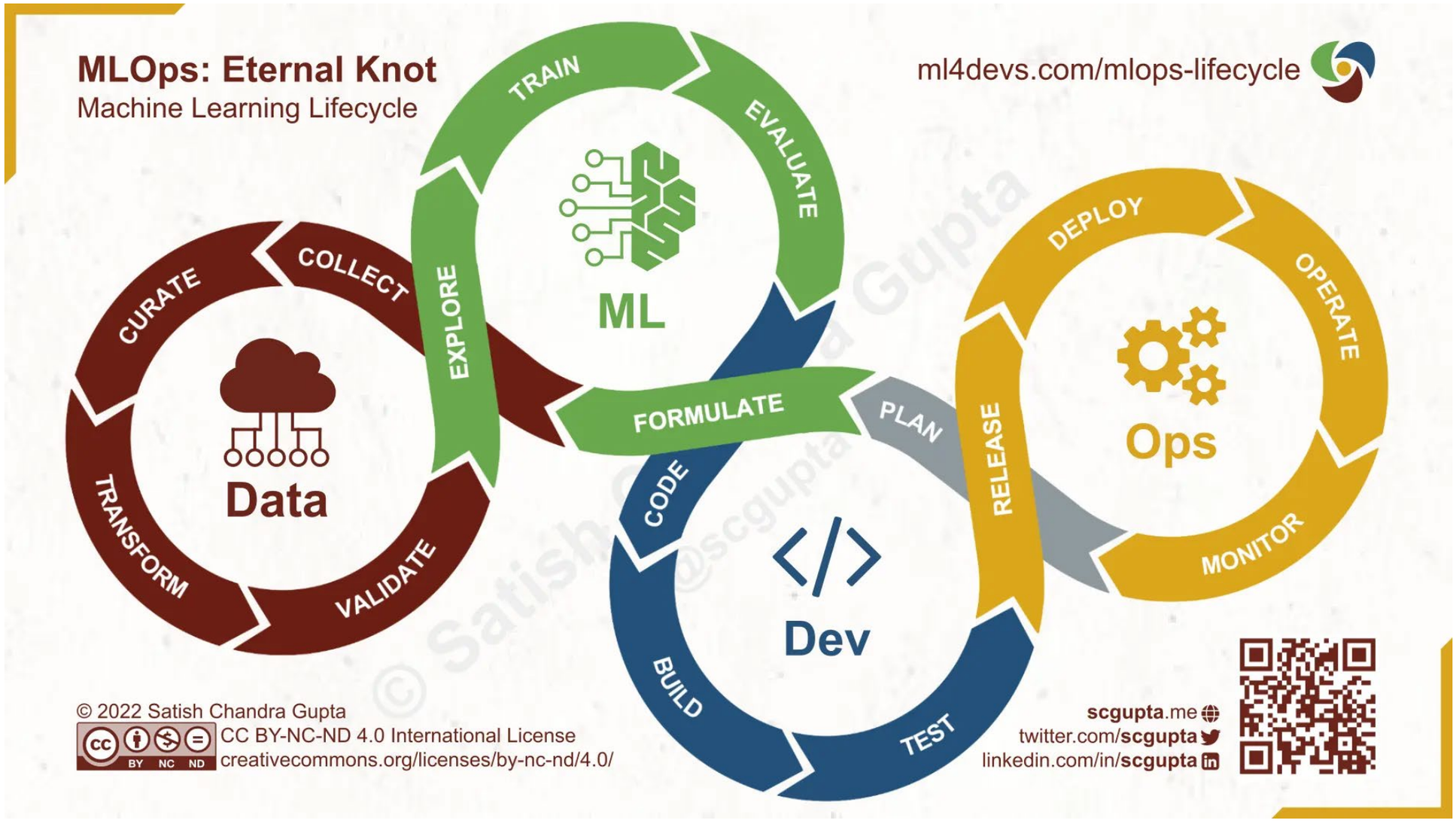
- Übersicht
- Aktivitätsprotokoll
- Zugriffssteuerung (IAM)

Durchsuchen | Beenden | Tauschen | Neu starten | Löschen | Aktualisieren | Veröffentlichungsprofil herunterladen

Zusammenfassung

Ressourcengruppe (...	: mdm-hikeplanner	Standarddomäne	: mdm-hikepla
Status	: Wird ausgeführt	App Service-Plan	: mdm-hikepla

Übersicht


MLOps: Eternal Knot
Machine Learning Lifecycle
ml4devs.com/mlops-lifecycle
TRAIN
EVALUATE
ML
CURATE
COLLECT
EXPLORE
Data
TRANSFORM
VALIDATE
FORMULATE
PLAN
CODE
Dev
BUILD
TEST
RELEASE
DEPLOY
OPERATE
Ops
MONITOR
© 2022 Satish Chandra Gupta
CC BY-NC-ND 4.0 International License
creativecommons.org/licenses/by-nc-nd/4.0/
scgupta.me
twitter.com/scgupta
linkedin.com/in/scgupta

Weiterführende Informationen

Wanderzeitformel

https://www.schweizer-wanderwege.ch/de/wissen/signalisation/wanderzeit#page-section-4

Idee

https://blog.mimacom.com/data-collection-scrapy-hiketime-prediction/

Code-Repository

https://github.com/mosazhaw/HikePlanner